

实验七 聚类算法

（一）目的和要求

- （1）理解聚类算法理论基础
- （2）掌握聚类算法的实现方法
- （3）理解并分析聚类方法的结果

（二）实验知识准备

聚类算法：聚类算法是在数据中发现数据对象之间的关系，将数据进行分组，使得组内的相似性尽可能大，而组间的差别尽可能大。聚类过程将物理或抽象对象的集合分成由类似对象组成的多个类或簇（cluster）。这些簇内的对象具有较高的相似度，而与其他簇的对象相似度较低。

（1）**K-Means 算法：**是一种非常流行的聚类算法，它属于无监督学习范畴，旨在将数据集分成 K 个簇，使得同一簇内的数据点尽可能相似，而不同簇之间的数据点差异尽可能大。

■ 基本步骤：

1. 从 N 个样本数据中随机选取 K 个对象作为初始的聚类中心。
2. 分别计算每个样本点到各个聚类中心的距离，并逐个分配到距离其最近的簇中。
3. 所有对象分配完成后，更新 K 个类中心位置，类中心定义为簇内所有对象在各个维度的均值。
4. 与前一次计算得到的 K 个聚类中心比较，如果聚类中心发生变化，则重复步骤 2，否则停止并输出聚类结果。

■ 关键问题：

1. 聚类个数 K 的选取。
2. 初始聚类中心点的选择。
3. 相似性与距离度量方法的选择。

（2）**层次聚类：**基本思想是将数据集构建成一个层次结构。在凝聚层次聚类（自下而上）中，每个样本最初表示为一个单独的簇，然后通过计算样本之间的相似度或距离来逐渐将簇合并成更大的簇，直到某个终结条件被满足。

整个过程可以表示为一棵树形结构，称为聚类树或树状图。通过该树状图，可以选择合适的切割点来确定最终的聚类结果。分裂层次聚类（自上而下）则相反，它首先将所有对象置于一个簇中，然后逐渐细分为越来越小的簇。

■ 基本步骤：

1. 初始化：将每个数据点视为一个单独的聚类。
2. 计算距离：计算所有聚类之间的距离。常用的距离度量包括欧氏距离、曼哈顿距离等。
3. 选择最近的聚类对：在所有聚类对中，选择距离最近的两个聚类进行合并。
4. 更新距离矩阵：合并选定的聚类对后，更新距离矩阵以反映新的聚类结构。这一步是凝聚层次聚类的关键，因为不同的距离更新策略会导致不同的聚类结果。
5. 重复步骤 2 至 4：直到达到指定的聚类数或所有数据点合并成一个聚类。
6. 生成树状图：通过记录每次合并的聚类 and 距离，生成一个树状图，用于可视化聚类过程。

■ 关键问题：

1. 聚类数的确定
2. 距离和相似度度量方法的选择
3. 噪声和异常值的处理
4. 计算复杂度和效率问题

（3）密度聚类（DBSCAN）：算法将具有足够高密度的区域划分为簇，并能在具有噪声的空间数据库中发现任意形状的簇。它将簇定义为密度相连的点的最大集合。算法的基本思想是通过检查数据点在其邻域内的密度来判断该点是否为核心点、边界点或噪声点。

■ 基本步骤：

1. 初始化：将所有点标记为未访问。
2. 迭代处理：对每个未访问的点，找到其 ϵ -邻域内的所有点。
3. 核心点检查：如果点的 ϵ -邻域内的点数大于等于 MinPts ，则将其标记为核心点，并创建一个新的簇。
4. 扩展簇：对簇中每个点，如果是核心点，将其 ϵ -邻域内的所有点加入簇

中并标记为已访问。重复此步骤，直到所有点都被访问。

5. 标记噪声点：未归入任何簇的点被视为噪声点。

■ 关键问题：

1. 参数选择，**eps**（邻域半径）：用于确定一个点的邻域范围。选择合适的**eps**对于算法的性能和聚类结果至关重要。**MinPts**（最小样本数）：用于定义核心点的阈值。**MinPts**的选择通常基于数据的维度和密度分布。

2. 参数敏感性，DBSCAN 算法对 **eps** 和 **MinPts** 的选择非常敏感。不同的参数组合可能导致完全不同的聚类结果。

(4) 距离和相似度度量方法：

■ 距离度量：

1. 欧式距离：定义在两个向量（两个点）上，是点 X 和 Y 之间的直线距离。

2. 曼哈顿距离：在二维空间中，两点之间的绝对距离之和。

3. 马氏距离：考虑数据分布特性的距离度量方法。

4. 汉明距离：用于等长字符串之间的不同字符个数。

■ 相似度度量：

1. 余弦相似度：通过计算两个向量的点积和向量长度来度量相似度。

2. 皮尔逊相关系数：用于度量两个变量之间的线性相关性。

（三）实验内容及步骤

(1) 使用 K-means 算法对鸢尾花（Iris）数据进行聚类

1. 步骤一，导入本项目所需要的模块和包，如图 1 所示。

```
1 import matplotlib.pyplot as plt
2 from sklearn.datasets import load_iris
3 from sklearn.cluster import KMeans
4 from sklearn.preprocessing import StandardScaler
```

图 1 导入所需模块

2. 步骤二，加载鸢尾花数据集并标准化数据。参考代码如图 2 所示。

```

1 iris = load_iris()
2 X = iris.data    # 特征数据
3 y = iris.target  # 实际标签（用于后续比较）
4 scaler = StandardScaler()
5 X_scaled = scaler.fit_transform(X)

```

图 2 从 Keras 加载数据集

3. 步骤三，使用“肘部法则”选择 K 值，主要代码如图 3 所示。

```

1 inertia = []
2 K_range = range(1, 11)
3
4 for k in K_range:
5     kmeans = KMeans(n_clusters=k, random_state=42)
6     kmeans.fit(X_scaled)
7     inertia.append(kmeans.inertia_)
8
9 # 绘制肘部图
10 plt.figure(figsize=(8, 4))
11 plt.plot(K_range, inertia, marker='o')
12 plt.title('Elbow Method for Optimal K')
13 plt.xlabel('Number of clusters (K)')
14 plt.ylabel('Inertia')
15 plt.grid()
16 plt.show()

```

图 3 选择 K 值

4. 步骤四，使用选择后的 K 值初始化分类器，示例代码如图 4 所示。

```

1 k = 3
2 kmeans = KMeans(n_clusters=k, random_state=42)
3 y_kmeans = kmeans.fit_predict(X_scaled)

```

图 4 使用选择后 K 值

“肘部法则”，如图 5 所示。

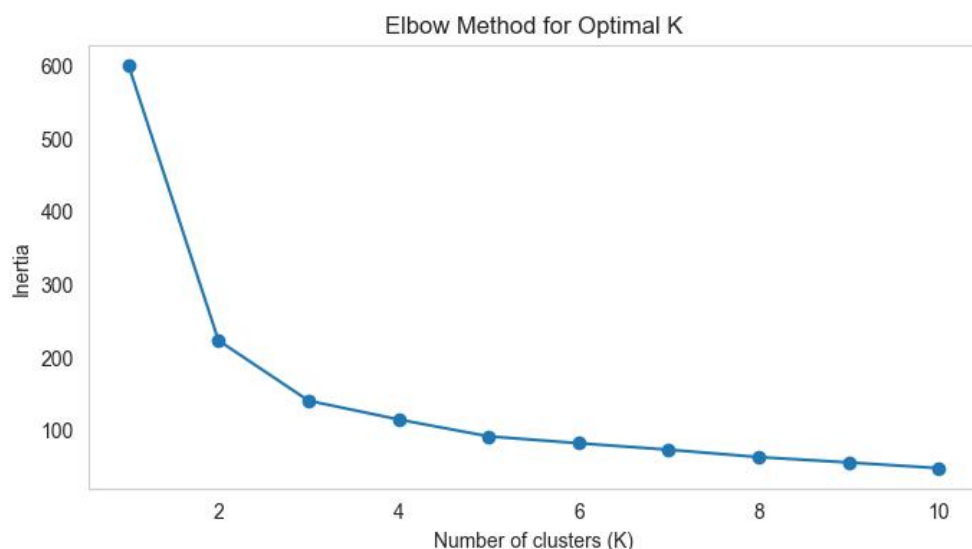


图 5 肘部法则

如上图所示：

A.横轴和纵轴

a.横轴（Number of clusters (K)）：表示选择的聚类数量 K，从 1 到 10。

b.纵轴（Inertia）：表示每个聚类数量对应的惯性（Inertia）即代价函数值（Cost Function Value），表示样本到其最近聚类中心的距离之和。惯性越小，表示聚类效果越好。

B.惯性值的变化

从 K=1 开始，随着 K 的增加，惯性值逐渐下降。这是因为增加聚类数量通常会减少每个聚类内部的距离，从而减小惯性。

C.肘部位置

观察图中，惯性值在 K=2 时急剧下降，随后在 K=3 到 K=6 之间变化较小。这个变化的拐点（即肘部）通常是选择最优 K 的位置。

本例中肘部大约出现在 K=3 到 K=4 之间。此时，增加 K 的收益开始减小。

D.聚类的选择

根据肘部法则，建议选择 K=3 或 K=4 作为最优聚类数，可以获得较好的聚类效果，同时避免过拟合（即选择过多的聚类）

5. 步骤五，绘制结果和聚类中心，示例代码如图 6。

```

1 plt.figure(figsize=(8, 6))
2 plt.scatter(X_scaled[y_kmeans == 0, 0], X_scaled[y_kmeans == 0, 1], s=100, c='red', label='Cluster 1')
3 plt.scatter(X_scaled[y_kmeans == 1, 0], X_scaled[y_kmeans == 1, 1], s=100, c='blue', label='Cluster 2')
4 plt.scatter(X_scaled[y_kmeans == 2, 0], X_scaled[y_kmeans == 2, 1], s=100, c='green', label='Cluster 3')
5
6 # 绘制聚类中心
7 plt.scatter(kmeans.cluster_centers[:, 0], kmeans.cluster_centers[:, 1], s=300, c='yellow', label='Centroids')
8 plt.title('K-means Clustering of Iris Data')
9 plt.xlabel('Feature 1 (scaled)')
10 plt.ylabel('Feature 2 (scaled)')
11 plt.legend()
12 plt.grid()
13 plt.show()

```

图 6 可视化结果

6. 步骤六，评价模型，示例代码如图 7 所示。

```

1 import seaborn as sns
2 from sklearn.metrics import confusion_matrix
3
4 confusion_mat = confusion_matrix(y, y_kmeans)
5 plt.figure(figsize=(8, 6))
6 sns.heatmap(confusion_mat, annot=True, fmt='d', cmap='Blues',
7             xticklabels=['Cluster 0', 'Cluster 1', 'Cluster 2'],
8             yticklabels=['Class 0', 'Class 1', 'Class 2'])
9 plt.title('Confusion Matrix')
10 plt.xlabel('Predicted Clusters')
11 plt.ylabel('True Classes')
12 plt.show()

```

图 7 评价模型

（2）实训项目

使用其他聚类方法对鸢尾花数据进行聚类分析，完成以下任务：

A.使用层次聚类对样本数据进行聚类，注意在层次聚类中阈值的选择与聚类结果的关系，可以使用聚类树来通过观察树的结构，手动选择合适的阈值，聚类树的结构如下：

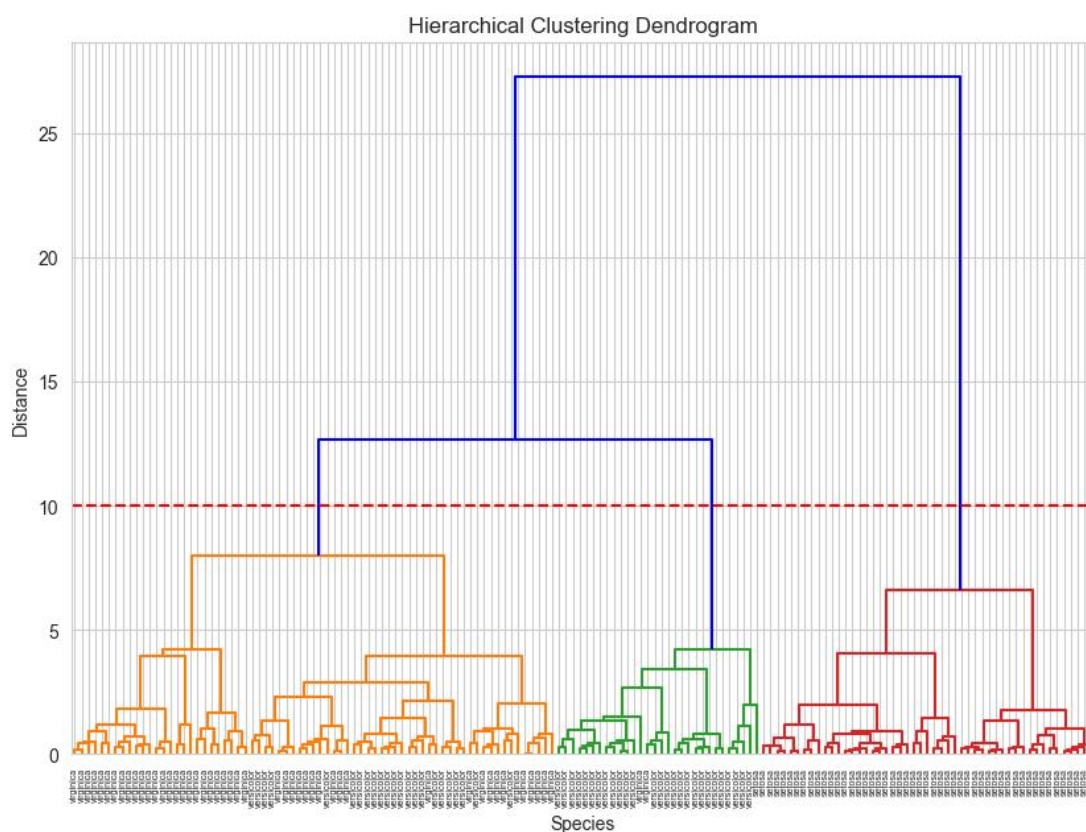


图 6 聚类树结构

如图 6 所示：

1.横轴和纵轴

c.横轴（Species）：表示数据点（在此情况下是鸢尾花的样本）。每个样本在树状图中都有一个标签。

d.纵轴（Distance）：表示聚类时的距离或相似度。距离越大，表示样本之间的相似度越低。

2.聚类结构

a.分支：每个分支代表一个聚类的合并过程。分支越高，表示合并时样本之间的距离越大。

b.颜色：不同颜色的分支（如橙色和绿色）表示不同的聚类。你可以看到，不同的物种在树中有明显的分离。

3.红色水平线（阈值）

a.阈值线：红色水平线表示选择的阈值。这个阈值用于决定聚类的数量。任何在此水平线之下的分支被认为是一个单独的聚类。

b.聚类数：如果将红线向左延伸，穿过树的分支，你可以看到有多少个聚类被形成。在此情况下，可能会形成 3 个主要的聚类，反映出鸢尾花的三种物种。

4.聚类的可视化

群体结构：从聚类树中可以看出，不同物种之间的相似性和差异性。例如，某些样本可能在特征上非常相似，导致它们被聚为同一类，而其他样本则被分到不同的类中。

5.聚类的选择

选择合适的阈值：根据你对实际类别（如鸢尾花的三种物种）的了解，可以选择一个合适的阈值，以确保最终聚类结果与实际情况相符。

B.使用密度聚类对样本数据进行聚类，注意参数选择方法：

a.Eps：选择合适的 **eps** 值至关重要。值过小会导致许多点被标记为噪声；值过大则可能将不同的聚类合并。可以通过绘制 **K-Distance** 图（**k-距离图**）来选择合适的 **eps** 值。

b.MinPts：这个参数定义了一个点成为核心点所需的最小邻居数量。值过小可能会导致噪声点被误认为是聚类的一部分，值过大可能会导致无法形成足够的聚类。